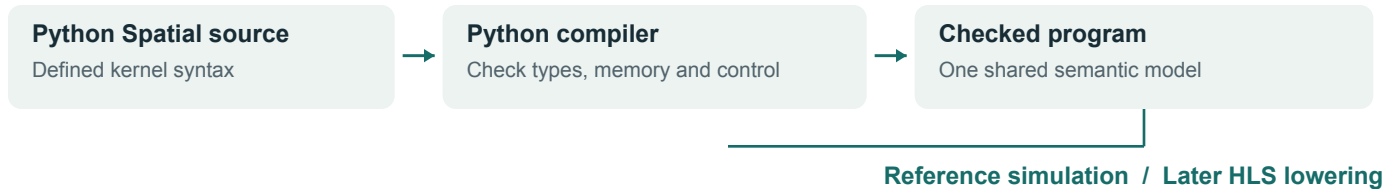


Python Spatial: the big picture

Research, architecture and the first implementation

The goal is a full Python implementation of Spatial: a Python frontend, compiler and reference simulator, followed by HLS lowering. The proposed architecture gives these stages one shared program model. We have moved from architecture research into the first compiler implementation, using EE 109 patterns as initial targets.

INTENDED ARCHITECTURE



Explore the work

[Research site](#)

Language design, feasibility studies and EE 109 mappings.

[Architecture map](#)

Excalidraw overview and detailed module responsibilities.

[Implementation progress and next steps](#)

Verified components, remaining work and the first Lab 1 target.

[Research repository](#)

Versioned studies, design decisions and specifications.

What the research supports

Python can express the required compiler algorithms. The design makes Spatial's numeric, memory and control rules explicit, so Python's own runtime behavior does not silently define the language. Kernels use a defined Python subset read as source; ordinary Python handles host data and experiments.

We propose immutable Python records as the shared program model, based on comparisons with xDSL on matched structured programs and edits. Source, generated programs and imported artifacts will use the same verifier. Simulation and later hardware planning will start from that checked meaning.

The next proof point is a complete family of composed memory programs. Full-workflow speed, memory use and hardware results still need to be measured.

What is built, and what comes next

Verified checkpoint: spatial-py 092637e | October 8, 2026

Implemented compiler components

Program structure	Immutable records, source provenance, type/layout rules, scalar-expression normalization and structural checks.
Numeric rules	Signed 32-bit data arithmetic, exact integer indices and Boolean rules, with independent value and fault cases.
Memory analysis	Tracks shared backing storage, views, helper borrowing and lifetimes; records the safety and initialization obligations still to resolve.
Branches and loops	Derives conservative scalar and control facts and required arithmetic checks, including nonzero divisors and loop steps. Runtime guards are not yet inserted.

At checkpoint 092637e, all 376 component tests passed from both the source tree and an isolated installed package. Independent Codex and Opus reviews identified issues in fault attribution and resource-limit diagnostics; fixes were reproduced and regression-tested. These results validate components, not a complete compiler.

Current boundary. The complete Lab 1 source-to-simulation path has not run yet. Full verification, source capture, input preparation and reference execution remain unfinished. No Python-generated HLS or hardware result is claimed.

Next steps

- 1 Finish the verifier.** Resolve safety conditions into proofs, supported runtime checks or errors; complete initialization and effect checks.
- 2 Connect source to execution.** Add Python source capture, input preparation and the reference simulator. Demonstrate the Lab 1 tiled-scale case and a composed helper/branch/tail variant against independent expected results.
- 3 Test generality, then expand.** Freeze the compiler and have an independent author supply a structurally unfamiliar program. Exercise a real structural edit, measure the full workflow, then extend language families and develop the HLS path from a validated subset.

What will count as success

The first milestone is a reusable family of memory programs. Changed shapes, nested loops, branches, helpers and aliases must use the same rules; no special path may recognize a lab or kernel. A successful example is evidence toward that milestone, not the entire acceptance test.